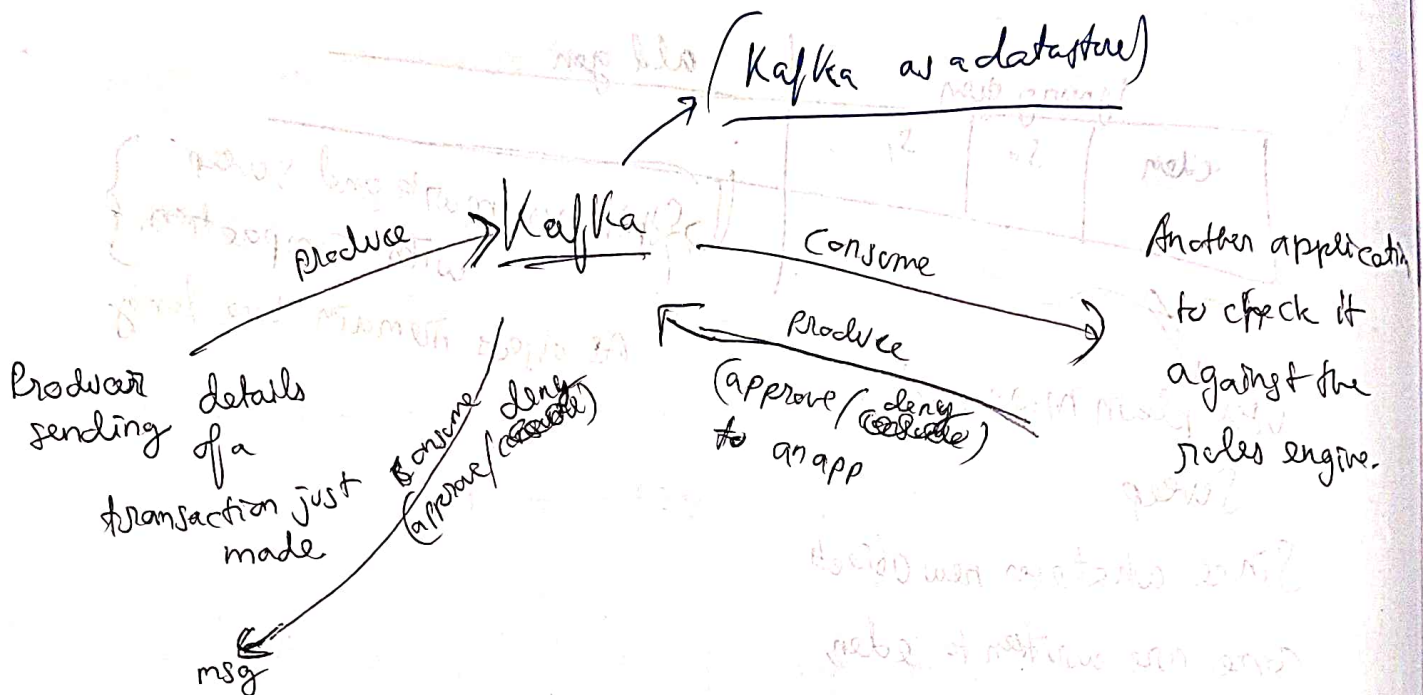


Kafka Producers, Writing messages to Kafka.

Chapter 3.



Apache Kafka ships APIs to make Kafka behave like a 'datastore'

(Intent)

* How to create `KafkaProducer` & `ProducerRecords` objects, how to handle errors that Kafka may return.

* Configuration options of producer

* Different partitioning methods and serializers and how to write own serial partitioners and serializers.

Third Party Clients.

Since Kafka is written in Java, and it does the process of (Key, Value) checksum check after decompressing and again recompresses it before storing it.

Kafka provides wire protocol, which is for non-Java applications, only bytes of data is transferred for them, ~~large~~ non-Java languages, like C++, python and Go can use this.

Producer Overview.

Credit card
use case

- Latency should be very low
- throughput high
- Cannot afford to loose messages or duplicate messages

tight constraints

Netflix clicks
use case

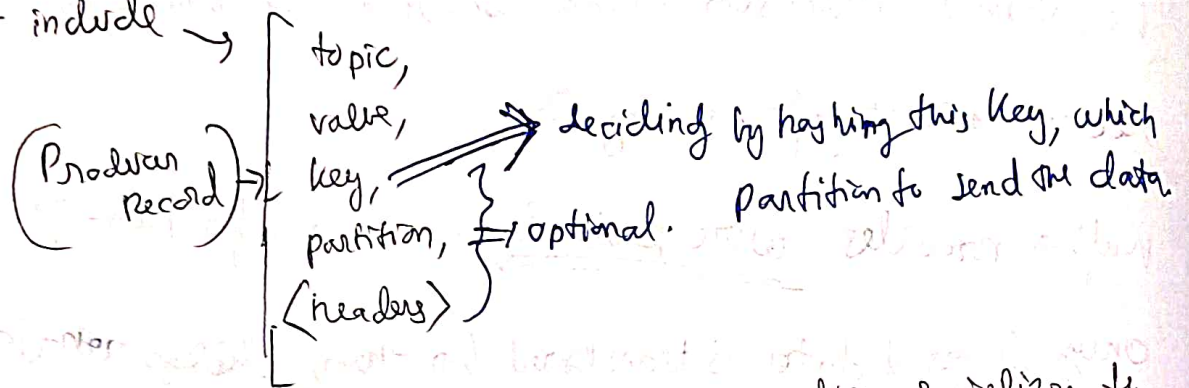
- duplication or losing some message is fine
- latency can be high
- throughput depending on access patterns.

Loose Constraints.

As per the requirements of our system, we can tune the producer and kafka configuration to cater to our needs.

How Producer actually sends the data?

- Producer produces messages to Kafka by creating a `ProducerRecord`, which must include



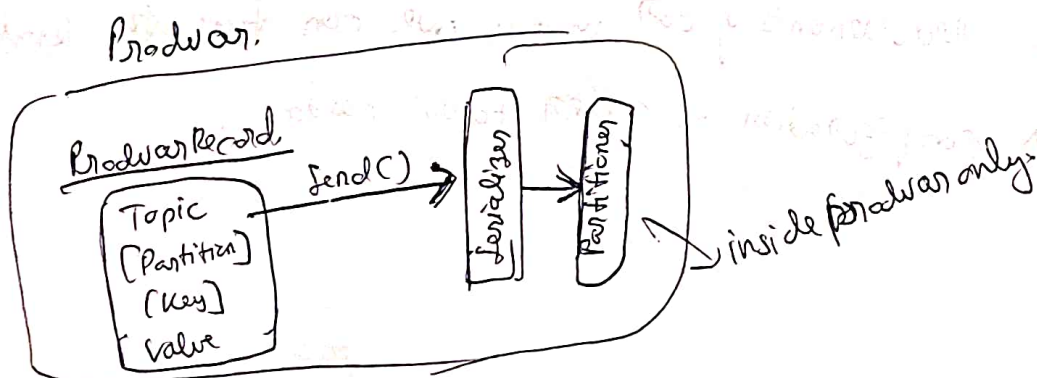
- When we hit the `producer.send(producerRecord)`, producer serializes the (key and value) objects to bytes to send it over network

- If we did not explicitly specify a partition, the data is sent to partitioner → ~~inside broker~~ ^{inside producer itself} - The partitioner will choose which partition to send data to. Once partition is selected, the producer knows which topic and partition record should go to.

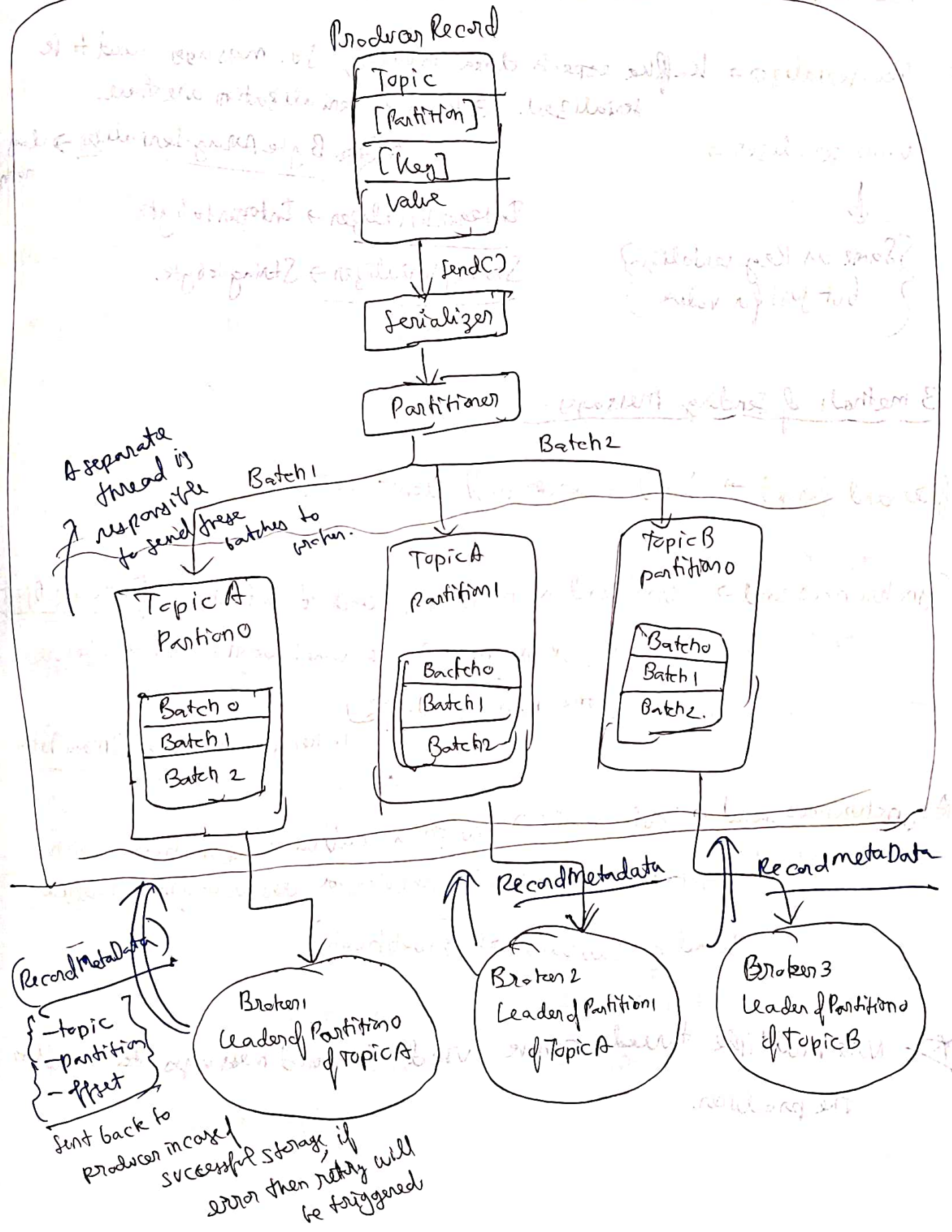
~~xxx~~ ^{import} Kafka producer client library includes a partition library itself

When the producer hits `send()`, the ~~partitioner~~ partitioner logic is invoked which decides which partition to send the msg,

Once the partition gets decided, message is sent to leader broker of that partition.



* One for a record the topic and partition is decided after partitioner, then these records are added to a batch of records that will be sent to same (partition & topic). (Producer)



Constructing a Kafka Producer.

- * bootstrap-servers → broker list of servers. (Kafka brokers).
- * key-serializer → Kafka expects data in bytes, so messages need to be serialized. 3 types of serialization are there.
- * value-serializer → Byte Array Serializer → ^{does nothing}
↓
{ Same as key serializer }
 but just for value
IntegerSerializer → Integer to byte
StringSerializer → String to byte

3 methods of sending messages.

fire and forget → send to server and don't care.

Synchronous send → we send a message, and it returns a Future Object.

However, we use get() to wait on the future, before sending the next record.

↘ To receive the record metadata.

Asynchronous send → call send() with a callback function, which gets triggered when it receives a response from Kafka.

Kind of callback url. {webhook}.

* Note multiple threads can be used to send messages ~~to the~~ from the producer.

Sending Message to Kafka

* `ProducerRecord <String, String> record = new ProducerRecord <>`
`( "abc", "def", "ghig" );`

Key ↑ Value ↑

public `ProducerRecord (String topic, K Key, V Value);`

Many other constructors are there

Before sending to partitioner.

ValueSerializer

KeySerializer

* `public ProducerRecord (String topic, Integer partition, Long timestamp, K Key, V Value, Iterable<Header> headers) {`

another constructor

* As we saw in the architecture diagram, ~~mess~~ producer record goes to Serializer and is placed in a buffer, which is sent to broker in a diff thread. • `send()` method returns a Java Future Object with Record Metadata,

`producer.send(record) →` Simply ignores the response, no retry mechanism.

* Though we don't do a get call to catch Exceptions, we might get errors if error occurred before sending to broker, maybe a SerializationException.

Sending a message Synchronously

Main tradeoff is performance,

Depending on how busy the Kafka clusters are Producers can take anywhere from 2ms to a few seconds to respond to `produce()` requests.

If you send synchronously

`producer.send(record).get()` → Producer will wait synchronously. and thread will wait doing nothing

(KafkaProducer has 2 types of Errors)

Retriable
Errors

Can be resolved by retry,
like a connection error, or a
node leader for a partition can get
fixed when request is hit again
after leader election is done again.

Non-Retriable
Errors

Some cannot be
fixed by retry,
like message size
is too big.

(P.T.O)

Sending Message Asynchronously.

In Kafka-clients library, there is a Callback() ~~is~~ Functional interface

```
package org.apache.kafka.clients.producer;
```

```
@FunctionalInterface  
public interface Callback {
```

```
    void onCompletion(RecordMetadata var1, Exception var2);
```

```
}
```

Code → {

```
    private class DemoProducerCallback implements Callback {
```

```
        @Override  
        public void onCompletion(RecordMetadata data, Exception e) {
```

```
            if (e != null) {  
                e.printStackTrace();
```

```
            }
```

```
        }
```

```
    }
```



Response from send()
method comes here

~~xxxx~~

Callbacks are executed in producer's main thread. The

messages sent to one partition one after another, their callbacks will be executed in the same order that we sent them. Because main thread.

This also means call back logic onCompletion() should be reasonably fast. If some processing of error needs to be done use a separate thread to do it concurrently.

Producer Configurations.

Client.id → Uniquely identifying a client/producer.

acks → this parameter controls how many partition replicas must receive the record before producer can consider the write successful.

default → Kafka will respond record written successfully after leader received the record. ~~leader responds~~ This option has significant impact on durability of messages.

acks=0 → Fire and forget. No wait

acks=1 → Leader's acknowledge.

acks=all → Leader + all in-sync replicas.

These options might make you think that based on these settings producer latency is less.

But overall latency from producer → (Kafka) → Consumer remains the same.

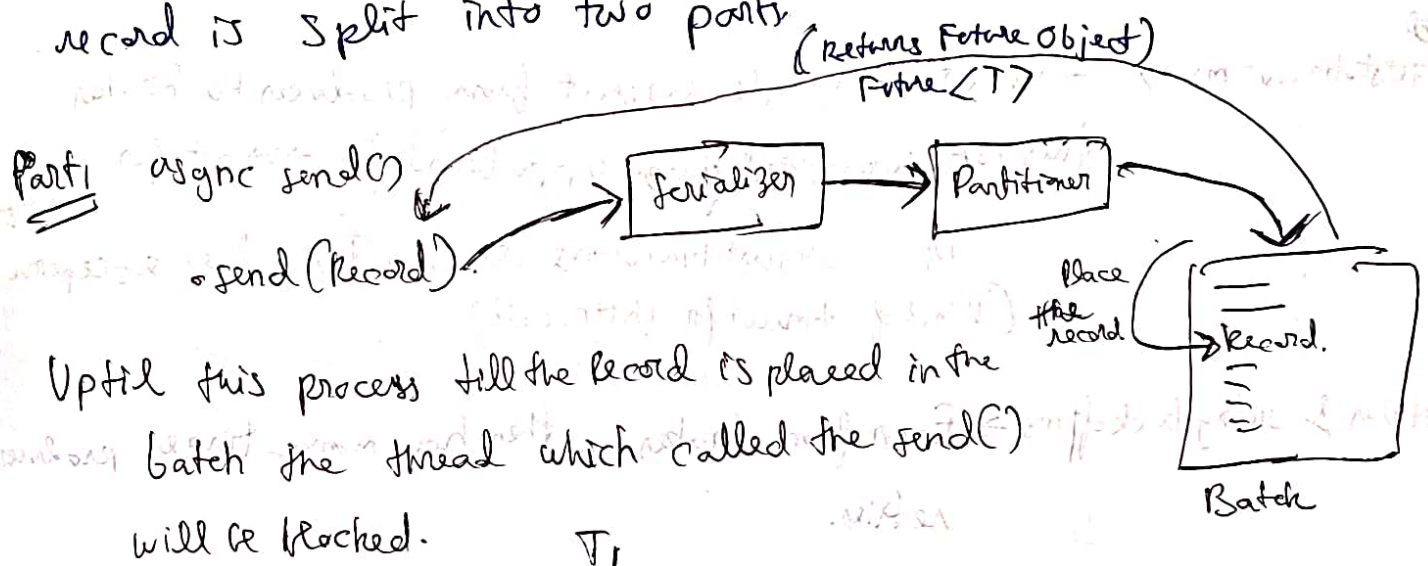
~~note~~

Kafka does not allow read to happen for consumers unless and until all in-sync replicas are getting the data.

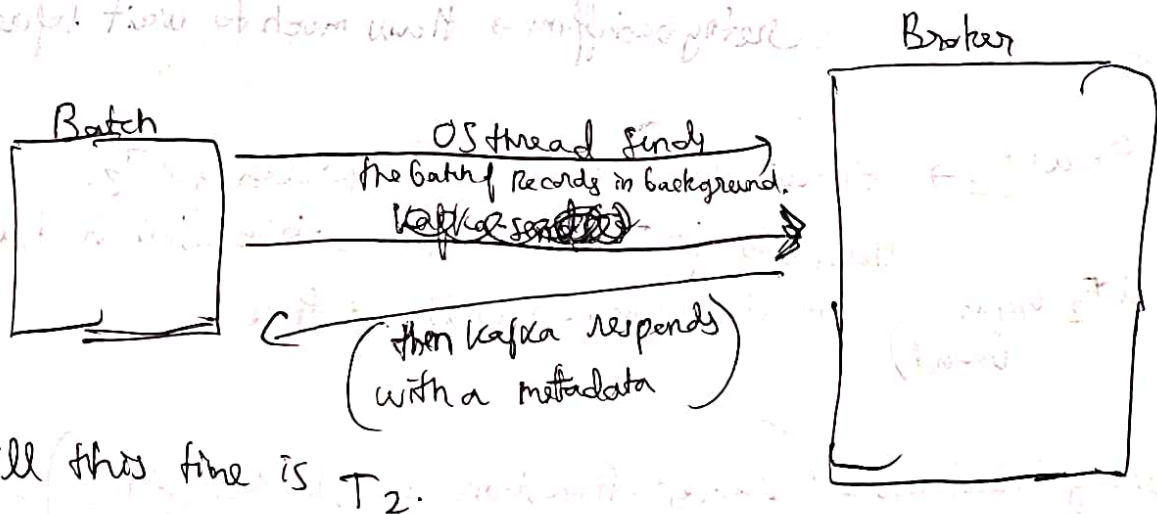
Message Delivery Time →

Configuration that interests Kafka developers most is that
→ how long will it take until a call to send() will succeed or fail. This is the time we are willing to spend until Kafka responds successfully or until we are willing to give up and admit defeat.

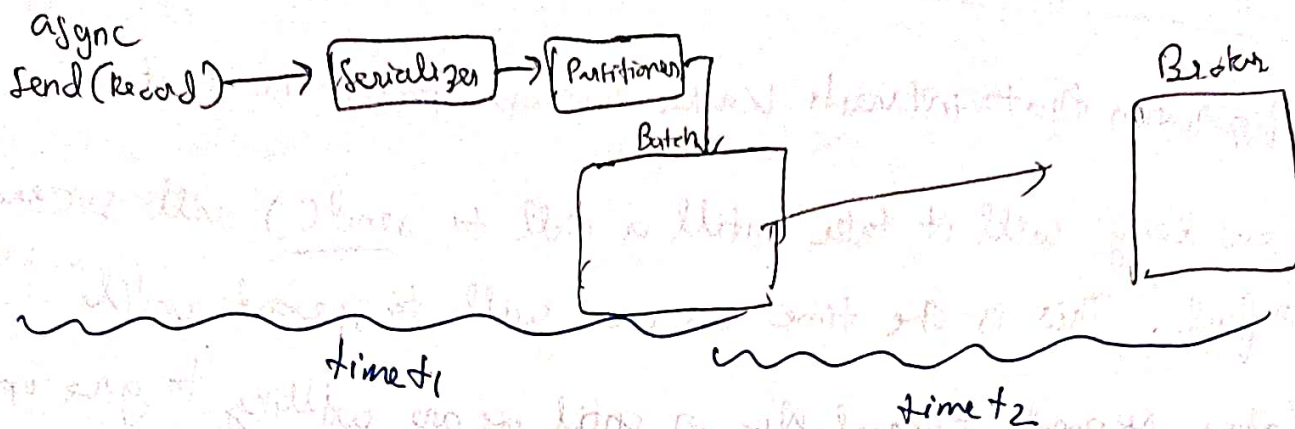
Since Apache Kafka, this time spent on sending a producer record is split into two parts.



Part 2



$$(T_1 + T_2 = \text{totalTime})$$



max-block.ms $\rightarrow$ this is the upper bound of T_1 .

if T_1 exceeds max-block.ms, then timeoutException.

~~do~~ request.timeout.ms $\rightarrow$ timeout for single request from producer to broker.

this is ~~upper bound of~~ ~~the~~ upper bound for timeout for single request

if request.timeout.ms exceeds T_2 , then exception (kind of timeout for http call).

retries & retry.backoff.ms $\rightarrow$ Error from broker, then how many times producer retries.

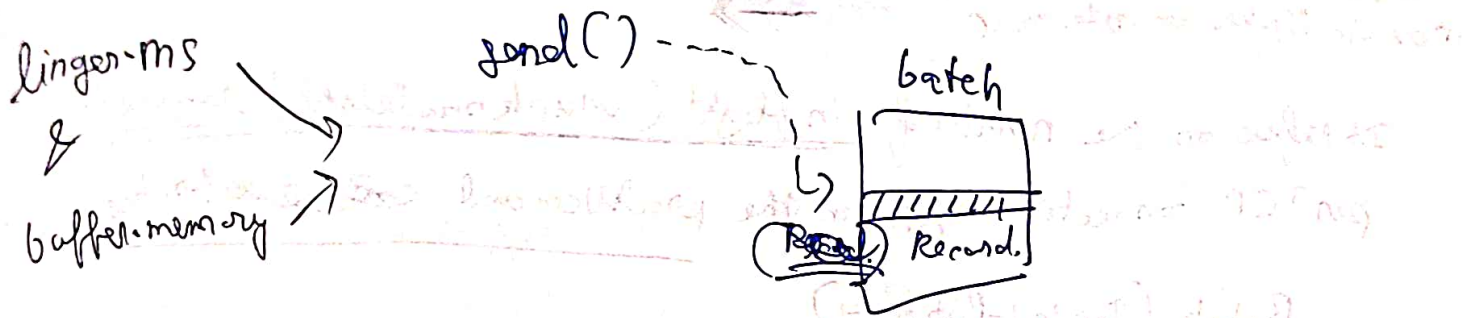
retry.backoff.ms $\rightarrow$ How much to wait before retrying.

delivery.timeout.ms $\rightarrow$ timeout is for the whole process for T_2 .

$\Downarrow$
(overall T_2 upper bound)

Upper limit for T_2 - Multiple retries will be there, multiple timeouts will be there.

$$\text{delivery.timeout.ms} = \left(\text{request.timeout.ms} + \underbrace{\text{retry.backoff.ms}}_{\text{if fails}} \right) \times \underbrace{\text{retries}}_{\text{if fails}}$$

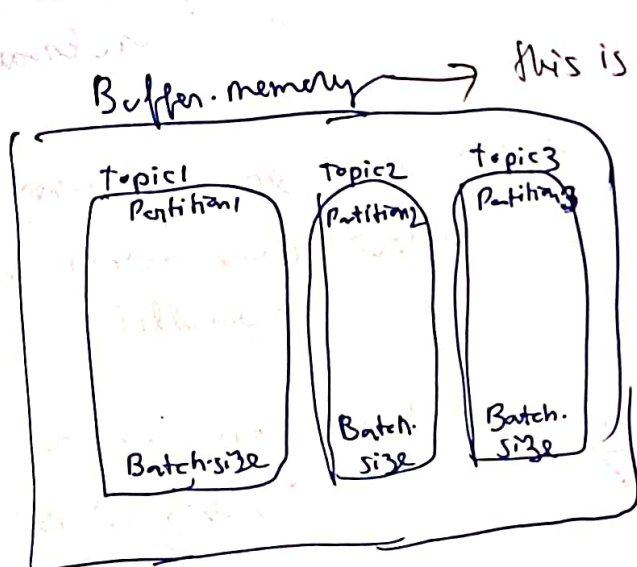


linger.ms, tells max time to wait before sending the batch to broker,

buffer.memory, tells max size of ~~batch~~ ^{RAM} after ~~batch~~ ^{batch} sent to broker.

~~whichever happens first is done. whichever reaches limit first decides when broker receives sent for batch.~~

Clear cut clarity b/w buffer.memory and batch.size.



this is the amount of memory allocated in RAM where the batches for topic-partition level are to be stored.

~~If messages are in disk, then loaded into memory.~~

Note this is not broker, this is on producer end.

when a send()
 send(record) call happens it checks if buffer.memory is full or not, if full, then it waits for max.block.ms.

Batch.size $\rightarrow$ Is the upper memory limit upon satisfying which the producer sends to broker and new empty Topic-Partition Batch is created.

if ~~buffer.size~~ buffer.memory = 32 MB and Batch.size = 16 KB,
 the total batches = $32 \text{ MB} / 16 \text{ KB} = 2000$.

max. in-flight requests per connection →

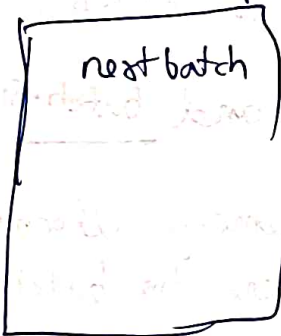
It refers to the number of in flight (unacknowledged) requests per TCP connection between the producer and ~~consumer~~ broker

Batch (Topic-Partition-1)



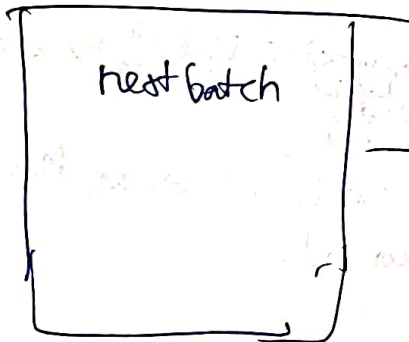
Sent over
Network

Batch (Topic-Partition-1)



Sent over
Network

Batch (Topic-Partition-1)



Sent over
Network

(If max. in flight requests per connection = 3)

then 3 batches can be sent without waiting for their acknowledgements

Higher value, ensures higher throughput (more batches sent in parallel).

But these can lead to ordering issues.

let's say

Batch1	→ sent	failed
Batch2	→ sent	processed.
Batch1	→ sent	processed.

But ordering got messed up.

Kafka Order Guarantee (within partition).

→ As we know Kafka guarantees writing and reading of messages within a partition in order. As long as producer sends them in order we are fine

But we saw in previous page an example where ordering got messed up. with

(retries and high max.in.flight.requests)

Fix for this?

enable.idempotence = true → this setting is a fine setting it tells you and explains to you how Kafka maintains deduplication and ordering of messages - Its so good that its fine and set into my brain

Step 1 → When you do enable.idempotence = true

Kafka producer starts assigning sequence numbers to records in the Batches.

Seq. No is per record and not per batch

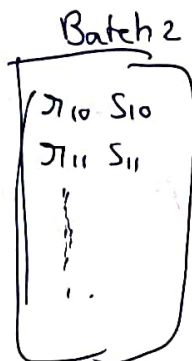
Sequence No are monotonically increasing

Batch 1	
Record 1	Seq 1
Record 2	Seq 2
Record 3	Seq 3
Record 4	Seq 4

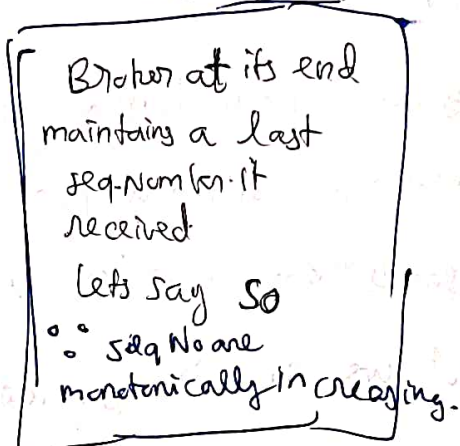
Batch 2	
Record 10	Seq 10
Record 11	Seq 11
Record 12	Seq 12
Record 13	Seq 13

(P.T.O)

Let's first ensure ordering handling.



Broker



Hence seq.No expected next = S_1

So our case, Batch 1 failed, but Batch 2 when tries to get persisted (π_{10}, S_{10}) (Does not match the expected seq No)

So Kafka rejects the batch 2, Yes! Exactly I was also shocked, Kafka actually rejects the batch and only process this batch when expect seqNo is S_{10} .

So when retry of Batch 1 happens

$(\pi_1, S_1), (\pi_2, S_2), (\pi_3, S_3), (\pi_4, S_4) \dots$

$(\pi_{10}, S_{10}), (\pi_{11}, S_{11}), (\pi_{12}, S_{12}) \dots$

processing done

(Broker)

this gets written over to the Broker,

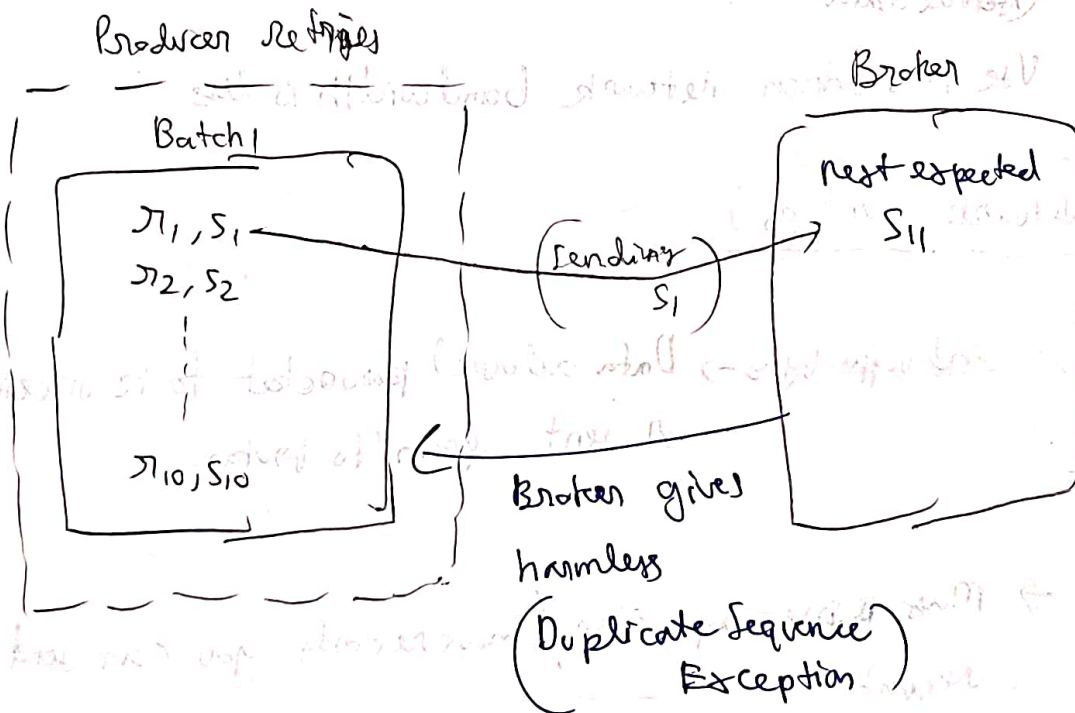
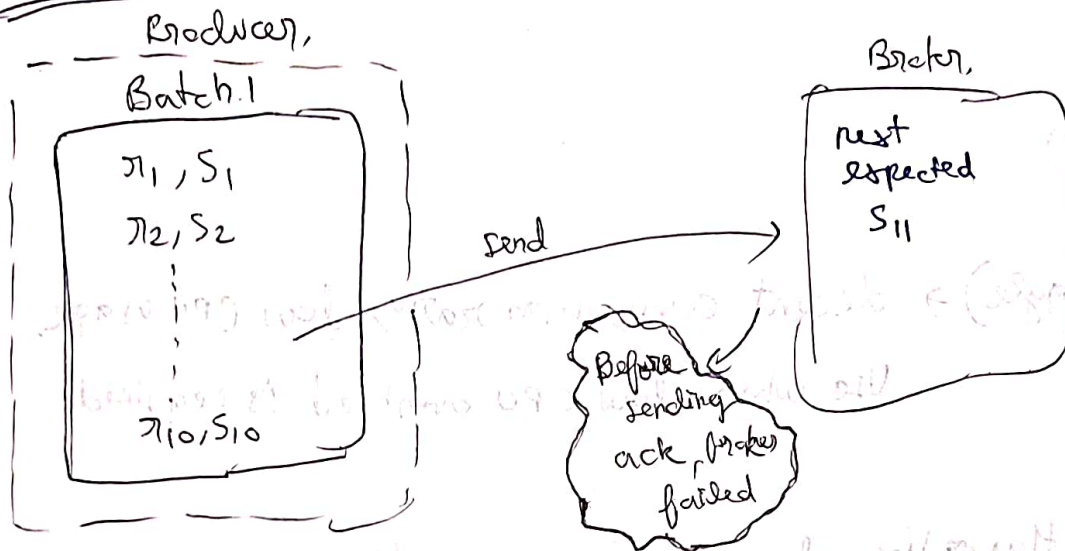
Now expecting S_{10} .

So for ordering don't accept ~~random~~ later records unless earlier records are processed.

Now let's come to deduplication.

When we again this Seq. No is only used to handle duplication.

Scenario



for (enable-idempotence=true) needs to have

(max.inflight.requests.per.connection ≤ 5), (retry > 0)

Compression-type

→ Compress messages to send over network to broker.
Compressing to send faster.

We know Kafka broker again decompresses does the checksum verification to find corrupt data and then recompresses to store in the broker.

(Compression types)

Snappy (Goggle) → decent compression ratio, low CPU usage.
Use when low CPU overhead is required.

GZip → Has a V-good compression ratio. But uses high CPU.
$$\left(\frac{\text{Compressed data}}{\text{Actual data}} \right) \approx \text{very low.}$$

Use this when network bandwidth is less.

(Some Network configs)

receive-buffer-bytes & send-buffer-bytes → Data allowed per socket to be received or sent from/to broker.

max-request-size → Max message size / max records you can send per request

↓

message-max-bytes → Config on broker, how many messages it shall receive.
↓
in bytes.

(Avro Serialisation & why it is good for Apache Kafka.)

Apache Avro is a data serialisation framework. With multiple benefits

- 1) Compact and Efficient → Encodes data in binary, and much more Compact ~~than JSON~~ and faster than JSON or XML.
- 2) Schema Based → You can provide default values.
- 3) Schema Evolution → Avro Supports forward & Backward Compatibility, will show using an example

Sample Schema (JSON format).

```
{ "type": "record"  
  "name": "Customer"  
  "namespace": "xyz"
```

```
  "fields": [  
    { "name": "id", "type": "int" },  
    { "name": "name", "type": "string" },
```

```
    { "name": "email", "type": ["null", "string"]
```

```
  ]  
}
```

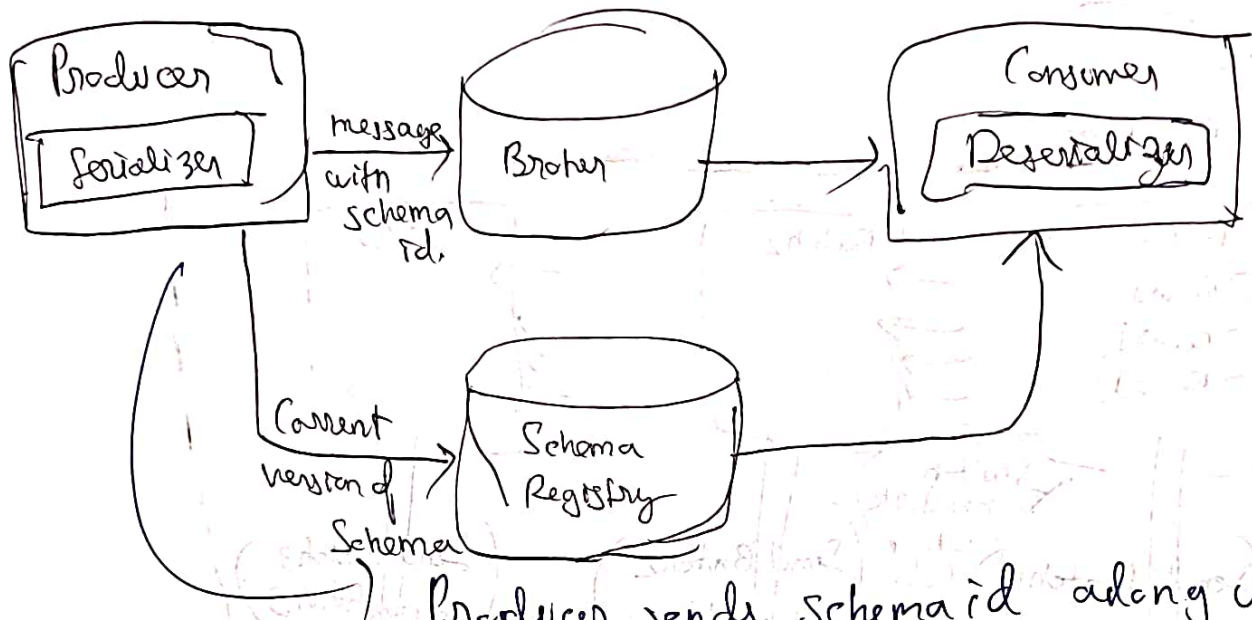
→ Schema Defined in

• avsc JSON file:

}, "default": "null" }

→ this default thing allows forward and backward compatibility

Apache Avro, keeps all its schema in a Schema Registry, which is being fetched by ~~the~~ ~~producer~~ consumer during deserialisation, and populated by producer during serialisation.



Producer sends schema id along with Avro records for it to fetch it from Schema Registry.

Instead of ~~up~~ sending schema for each record making the message bulkier.

(Kafka Headers)

Kafka message looks like

```

{ Key: "k1",
  Value: "-----v1",
  headers: {
    "app-version": "1.4.2",
    "content-type": "json",
    "source": "web"
  }
}
  
```

Broker does not care abt headers only producer and consumer utilize it.

Some Use-Cases.

→ Versioning: We studied abt avro right- Schema registry getting used for versioning

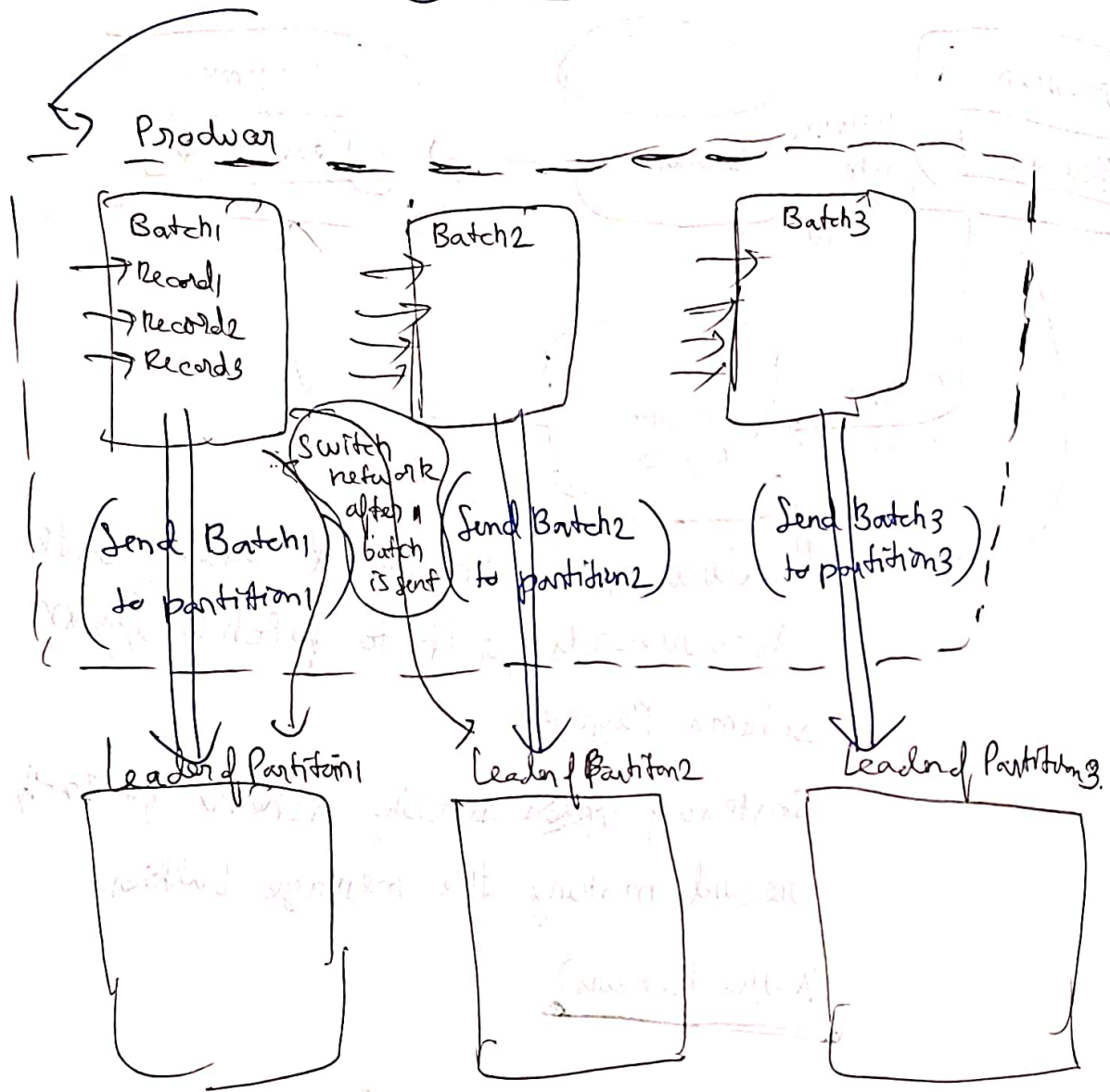
→ Some routing logic as well.

```

ProducerRecord(String, String) record = new ProducerRecord<>
    ("topicName", "k1", "v1");
record.headers().add("schema-version", "v2");
  
```

Partitioning

if no ~~partitioning~~ partitioning logic is there, by default Kafka does a sticky - round robin.



if key is given to record,
the $(\text{hash}(\text{key}) \% \text{no of partitions})$.

if we change no of partitions, then keys need to be remapped,
its better to decide on number of keys before hand.

{CustomPartitioner}

public class CustomPartitioner implements Partitioner

public void configure (Map<String, ?> configs) { }

@Override

public int partition (String topic, Object key, byte[] keybytes, Object value, byte[] valuebytes, Cluster cluster) {
 method
 { to assign partition number.

if (!keybytes == null || (!key instanceof String)) {

3 } send ("key needs to be there");

if (key.endsWith ("Cristiano Ronaldo")) {

3 return ~~topic~~ partitioner.size() - 1;

}

3

Send all CR7 messages to last partition.

Getting Partitions for topic. List<PartitionInfo> partitions = cluster.getPartitionsForTopic (topic);

(Quotas & Throttling)

Config

quota-producer-default = 2M

quota-consumer-default = 3M

is by default static and needs server restart to change, but using CLI can be made dynamic.

(max writes per sec)
 2MB/s
 (max reads per sec)
 3MB/s

a) What broker does if producer/consumer goes beyond quota?

A) Broker throttles it and delays the RecordMetadata response for send() call, so until the ack producer can't send new batch. Only (max-in-flight-msg) allowed